SQL and Python Event Generation

https://gemini.google.com/app/3689f20e3602f4ae

User prompt: help me ceat events for this data base table 1. Users user_id name role 1 Maria Lopez student 2 Daniel Garcia student 3 Sofia Martinez student 4 Carlos Rivera driver 5 Jose Hernandez driver 2. Sellers seller_id store_name category 1 Campus Coffee food 2 UCen Market food 3 Campus Bookstore bookstore 3. Items item_id seller_id name price is_available 101 1 Iced Coffee $4.50 TRUE 102 1 Blueberry Muffin $3.25 TRUE 103 2 Chicken Sandwich $8.99 TRUE 104 2 Bottled Water $2.00 TRUE 105 3 Notebook $5.50 TRUE 4. Orders order_id student_id driver_id delivery_location order_status estimated_delivery_time created_at 1001 1 4 Library delivered 09:15 08:30 1002 2 NULL Student Housing cancelled 12:45 12:10 1003 3 5 Engineering Building delivered 17:45 17:00 All timestamps are on November 2, 2026.5. Order Items order_item_id order_id item_id quantity 1 1001 101 1 2 1001 103 2 3 1001 104 1 4 1002 102 2 5 1003 105 1 6. Events This is the most important table for your event-generation model. event_id order_id seller_id event_type timestamp 1 1001 NULL OrderStarted 08:30 2 1001 1 AddItem 08:32 3 1001 2 AddItem 08:34 4 1001 NULL OrderPlaced 08:38 5 1001 NULL PaymentCompleted 08:44 6 1001 1 ItemsPreparing 08:46 7 1001 2 ItemsPreparing 08:47 8 1001 NULL DriverAssigned 08:48 9 1001 1 ItemsReady 08:53 10 1001 2 ItemsReady 08:55 11 1001 1 ItemsPickedUp 09:02 12 1001 2 ItemsPickedUp 09:04 13 1001 NULL OrderDelivered 09:15 14 1002 NULL OrderStarted 12:10 15 1002 1 AddItem 12:12 16 1002 NULL OrderPlaced 12:18 17 1002 NULL PaymentFailed 12:24 18 1002 NULL OrderCancelled 12:26 19 1003 NULL OrderStarted 17:00 20 1003 3 AddItem 17:03 21 1003 NULL OrderPlaced 17:08 22 1003 NULL PaymentCompleted 17:14 23 1003 3 ItemsPreparing 17:16 24 1003 NULL DriverAssigned 17:18 25 1003 3 ItemsReady 17:23 26 1003 3 ItemsPickedUp 17:31 27 1003 NULL OrderDelivered 17:45 7. Summary of relationships Order Student Sellers involved Payment Final outcome 1001 Maria Lopez Campus Coffee, UCen Market Successful Delivered 1002 Daniel Garcia Campus Coffee Failed Cancelled 1003 Sofia Martinez Campus Bookstore Successful Delivered

Response: For generating event streams for orders like these, SQL `INSERT` statements or a Python synthetic data generation script are the standard approaches.

Here is the complete SQL script to populate the `Events` table matching your target dataset, followed by a reusable Python generator for creating additional synthetic event sequences.

1. SQL Insert Script (`Events` Table)

```
INSERT INTO Events (event_id, order_id, seller_id, event_type, timestamp) VALUES
-- Order 1001 (Multi-seller: Campus Coffee [1] & UCen Market [2])
(1, 1001, NULL, 'OrderStarted', '2026-11-02 08:30:00'),
(2, 1001, 1, 'AddItem', '2026-11-02 08:32:00'),
(3, 1001, 2, 'AddItem', '2026-11-02 08:34:00'),
(4, 1001, NULL, 'OrderPlaced', '2026-11-02 08:38:00'),
(5, 1001, NULL, 'PaymentCompleted', '2026-11-02 08:44:00'),
(6, 1001, 1, 'ItemsPreparing', '2026-11-02 08:46:00'),
(7, 1001, 2, 'ItemsPreparing', '2026-11-02 08:47:00'),
(8, 1001, NULL, 'DriverAssigned', '2026-11-02 08:48:00'),
(9, 1001, 1, 'ItemsReady', '2026-11-02 08:53:00'),
(10, 1001, 2, 'ItemsReady', '2026-11-02 08:55:00'),
(11, 1001, 1, 'ItemsPickedUp', '2026-11-02 09:02:00'),
(12, 1001, 2, 'ItemsPickedUp', '2026-11-02 09:04:00'),
(13, 1001, NULL, 'OrderDelivered', '2026-11-02 09:15:00'),

-- Order 1002 (Failed Payment / Cancelled)
(14, 1002, NULL, 'OrderStarted', '2026-11-02 12:10:00'),
(15, 1002, 1, 'AddItem', '2026-11-02 12:12:00'),
(16, 1002, NULL, 'OrderPlaced', '2026-11-02 12:18:00'),
(17, 1002, NULL, 'PaymentFailed', '2026-11-02 12:24:00'),
(18, 1002, NULL, 'OrderCancelled', '2026-11-02 12:26:00'),

-- Order 1003 (Single-seller: Campus Bookstore [3])
(19, 1003, NULL, 'OrderStarted', '2026-11-02 17:00:00'),
(20, 1003, 3, 'AddItem', '2026-11-02 17:03:00'),
(21, 1003, NULL, 'OrderPlaced', '2026-11-02 17:08:00'),
(22, 1003, NULL, 'PaymentCompleted', '2026-11-02 17:14:00'),
(23, 1003, 3, 'ItemsPreparing', '2026-11-02 17:16:00'),
(24, 1003, NULL, 'DriverAssigned', '2026-11-02 17:18:00'),
(25, 1003, 3, 'ItemsReady', '2026-11-02 17:23:00'),
(26, 1003, 3, 'ItemsPickedUp', '2026-11-02 17:31:00'),
(27, 1003, NULL, 'OrderDelivered', '2026-11-02 17:45:00');
```

2. Python Generator Script

If you need to automatically synthesize events for new orders based on `Order_Items`, this script simulates realistic timestamp progression for both successful and cancelled flows:

```
from datetime import datetime, timedelta
import random

def generate_order_events(order_id, seller_ids, start_time_str, is_success=True, start_event_id=1):
    events = []
    curr_id = start_event_id
    curr_time = datetime.strptime(f"2026-11-02 {start_time_str}", "%Y-%m-%d %H:%M")

    # 1. Start Order
    events.append((curr_id, order_id, None, "OrderStarted", curr_time.strftime("%H:%M")))
    curr_id += 1

    # 2. Add Items per Seller
    for seller_id in set(seller_ids):
        curr_time += timedelta(minutes=random.randint(1, 3))
        events.append((curr_id, order_id, seller_id, "AddItem", curr_time.strftime("%H:%M")))
        curr_id += 1

    # 3. Place Order
    curr_time += timedelta(minutes=random.randint(3, 6))
    events.append((curr_id, order_id, None, "OrderPlaced", curr_time.strftime("%H:%M")))
    curr_id += 1

    if not is_success:
        # Failed payment workflow
        curr_time += timedelta(minutes=random.randint(4, 7))
        events.append((curr_id, order_id, None, "PaymentFailed", curr_time.strftime("%H:%M")))
        curr_id += 1
        curr_time += timedelta(minutes=random.randint(1, 3))
        events.append((curr_id, order_id, None, "OrderCancelled", curr_time.strftime("%H:%M")))
        return events

    # 4. Successful Payment
    curr_time += timedelta(minutes=random.randint(4, 7))
    events.append((curr_id, order_id, None, "PaymentCompleted", curr_time.strftime("%H:%M")))
    curr_id += 1

    # 5. Items Preparing
    for seller_id in set(seller_ids):
        curr_time += timedelta(minutes=random.randint(1, 2))
        events.append((curr_id, order_id, seller_id, "ItemsPreparing", curr_time.strftime("%H:%M")))
        curr_id += 1

    # 6. Driver Assigned
    curr_time += timedelta(minutes=random.randint(1, 3))
    events.append((curr_id, order_id, None, "DriverAssigned", curr_time.strftime("%H:%M")))
    curr_id += 1

    # 7. Items Ready
    for seller_id in set(seller_ids):
        curr_time += timedelta(minutes=random.randint(5, 8))
        events.append((curr_id, order_id, seller_id, "ItemsReady", curr_time.strftime("%H:%M")))
        curr_id += 1

    # 8. Items Picked Up
    for seller_id in set(seller_ids):
        curr_time += timedelta(minutes=random.randint(6, 10))
        events.append((curr_id, order_id, seller_id, "ItemsPickedUp", curr_time.strftime("%H:%M")))
        curr_id += 1

    # 9. Order Delivered
    curr_time += timedelta(minutes=random.randint(10, 15))
    events.append((curr_id, order_id, None, "OrderDelivered", curr_time.strftime("%H:%M")))

    return events
```

User prompt: can you do it in trs forma BASE_TIME_GRANULARITY = 1 MINUTE START = 0, END = 12 HOUR RESOURCES TABLES Stations, Users, Bikes, END TABLES VARIABLES Map_bounds = ([0,100],[0,100]), END VARIABLES FUNCTIONS city_block_distance, END FUNCTIONS CREATE EVENTTYPE RentBike (EVENTTIME 1, INGESTTIME (1) MAXDELAY (10), user_id: User(id) FOREIGNKEY, bike_id: Bike(id) FOREIGNKEY, session_id: AUTO_INCREMENT, station_id: Station(id) FOREIGNKEY, station_name: Station(name), location_data: Station(longitude,latitude), credit_card_num:User(credit_card_num) , is_member: User(is_member) , session_id -> ALL) CREATE EventType ReturnBike (EVENTTIME 1, INGESTTIME (1) MAXDELAY (10), user_id: INHERIT RentBike(user_id), bike_id: INHERIT RentBike(bike_id), station_id: Station(station_id) FOREIGNKEY, session_id: INHERIT RentBike(session_id) FOREIGNKEY, station_name: Station(station_name), location_data: Station(longitude,latitude), credit_card_num: INHERIT RentBike(credit_card_num), is_member: INHERIT RentBike(is_member), session_id -> ALL) CREATE EventType ReportLocation (EVENTTIME 1, INGESTTIME (1) MAXDELAY (10), bike_id: INHERIT RentBike(bike_id), session_id: INHERIT RentBike(session_id) FOREIGNKEY, user_id: INHERIT RentBike(user_id), is_member: INHERIT RentBike(is_member), location_data: IN Grid_map, session_id -> (bike_id, user_id, is_member)) E0: CAUSE START; RentBike E1: CAUSE RentBike; ReturnBike TIME BETWEEN 1min AND 24HOUR E2: CAUSE RentBike; ReportLocation TIME EXACTLY 30MINUTE E3: CAUSE ReportLocation; ReportLocation TIME EXACTLY 1HOUR TERMINATE ReturnBike CREATE DISTRIBUTION FOR START (BASE_TIME_GRANULARITY = 1 HOUR, [0,8) : 500, [8,9) : 800, [9,10) : 740, [10,11) : 740, [11,12) : 800) CREATE TIME DISTRIBUTION FOR E1 (BASE_TIME_GRANULARITY = 1 HOUR, [0,1) : 500, [1,2) : 800, [2,3) : 940, [3,4) : 650, [4,5) : 300)

Response: Here is your campus delivery dataset converted into the target TRS (Time-Series / Event Generation DSL) format.

```
BASE_TIME_GRANULARITY = 1 MINUTE
START = 0
END = 12 HOUR

RESOURCES
TABLES
Users, Sellers, Items, Orders, OrderItems
END TABLES
VARIABLES
Map_bounds = ([0,100],[0,100])
END VARIABLES
FUNCTIONS
city_block_distance
END FUNCTIONS

-- EVENT TYPES --

CREATE EVENTTYPE OrderStarted (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: Order(order_id) FOREIGNKEY,
       student_id: User(user_id) FOREIGNKEY,
       delivery_location: Order(delivery_location),
       order_id -> ALL
)

CREATE EVENTTYPE AddItem (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: INHERIT OrderStarted(order_id) FOREIGNKEY,
       seller_id: Seller(seller_id) FOREIGNKEY,
       item_id: Item(item_id) FOREIGNKEY,
       quantity: OrderItem(quantity),
       order_id -> ALL
)

CREATE EVENTTYPE OrderPlaced (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: INHERIT OrderStarted(order_id) FOREIGNKEY,
       student_id: INHERIT OrderStarted(student_id),
       order_id -> ALL
)

CREATE EVENTTYPE PaymentCompleted (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: INHERIT OrderStarted(order_id) FOREIGNKEY,
       order_id -> ALL
)
```

```
CREATE EVENTTYPE PaymentFailed (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: INHERIT OrderStarted(order_id) FOREIGNKEY,
       order_id -> ALL
)

CREATE EVENTTYPE ItemsPreparing (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: INHERIT OrderStarted(order_id) FOREIGNKEY,
       seller_id: Seller(seller_id) FOREIGNKEY,
       order_id -> ALL
)

CREATE EVENTTYPE DriverAssigned (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: INHERIT OrderStarted(order_id) FOREIGNKEY,
       driver_id: User(user_id) FOREIGNKEY,
       order_id -> ALL
)

CREATE EVENTTYPE ItemsReady (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: INHERIT OrderStarted(order_id) FOREIGNKEY,
       seller_id: Seller(seller_id) FOREIGNKEY,
       order_id -> ALL
)

CREATE EVENTTYPE ItemsPickedUp (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: INHERIT OrderStarted(order_id) FOREIGNKEY,
       seller_id: Seller(seller_id) FOREIGNKEY,
       driver_id: INHERIT DriverAssigned(driver_id),
       order_id -> ALL
)

CREATE EVENTTYPE OrderDelivered (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: INHERIT OrderStarted(order_id) FOREIGNKEY,
       driver_id: INHERIT DriverAssigned(driver_id),
       order_id -> ALL
)

CREATE EVENTTYPE OrderCancelled (
       EVENTTIME 1,
       INGESTTIME (1) MAXDELAY (5),
       order_id: INHERIT OrderStarted(order_id) FOREIGNKEY,
       order_id -> ALL
)

-- EVENT FLOW CAUSAL CHAINS --

-- Success Workflow
E0: CAUSE START; OrderStarted
E1: CAUSE OrderStarted; AddItem TIME BETWEEN 2MINUTE AND 5MINUTE
E2: CAUSE AddItem; OrderPlaced TIME BETWEEN 4MINUTE AND 8MINUTE
E3: CAUSE OrderPlaced; PaymentCompleted TIME BETWEEN 5MINUTE AND 10MINUTE
E4: CAUSE PaymentCompleted; ItemsPreparing TIME BETWEEN 2MINUTE AND 5MINUTE
E5: CAUSE ItemsPreparing; DriverAssigned TIME BETWEEN 1MINUTE AND 3MINUTE
E6: CAUSE ItemsPreparing; ItemsReady TIME BETWEEN 5MINUTE AND 15MINUTE
E7: CAUSE ItemsReady; ItemsPickedUp TIME BETWEEN 5MINUTE AND 10MINUTE
E8: CAUSE ItemsPickedUp; OrderDelivered TIME BETWEEN 10MINUTE AND 20MINUTE
```

```
TERMINATE OrderDelivered

-- Failure / Cancellation Workflow
E9: CAUSE OrderPlaced; PaymentFailed TIME BETWEEN 5MINUTE AND 10MINUTE
E10: CAUSE PaymentFailed; OrderCancelled TIME BETWEEN 1MINUTE AND 3MINUTE
TERMINATE OrderCancelled

-- DISTRIBUTIONS --

CREATE DISTRIBUTION FOR START (
       BASE_TIME_GRANULARITY = 1 HOUR,
       [0,8) : 10,
       [8,10) : 150,   -- Morning rush (e.g., Order 1001)
       [10,12) : 80,
       [12,14) : 300,  -- Lunch rush (e.g., Order 1002)
       [14,17) : 100,
       [17,19) : 250   -- Dinner rush (e.g., Order 1003)
)

CREATE TIME DISTRIBUTION FOR E8 (
       BASE_TIME_GRANULARITY = 1 MINUTE,
       [10,15) : 400,  -- On-campus nearby delivery
       [15,20) : 600,  -- Standard delivery time
       [20,30) : 100   -- Delayed delivery
)
```
